

TrustScientist: Evidence and Branch Revalidation for Autonomous Research Agents

Anonymous Author(s)

Abstract

A current evidence certificate can leave an autonomous research agent following a stale decision. TrustScientist addresses this gap with Counterfactual Branch Revalidation (CBR). It records the inputs and policy context of each decision, replays historical decisions affected by an evidence update, and restores the checkpoint before the earliest changed action. Versioned certificates maintain artifact checks; decision receipts maintain the branch history. We test CBR on controlled updates to real CORE-Bench artifacts and on executed, scripted analysis histories with model fitting after rollback. CBR matches full replay’s rollback points and final output consistency, with 35.3% fewer post-update policy evaluations in the latter study. Balanced timing shows a smaller saving. A generic dependency-replay implementation matches correctness and uses fewer evaluations than CBR, locating the benefit in selective decision replay rather than a unique graph representation. Separate acquisition and semantic studies retain strong-baseline ties, binding errors and failed risk calibration. Under deterministic policies and declared dependencies, CBR maintains a decision history consistent with updated evidence; natural autonomous histories and scientific policy validity remain outside this guarantee.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**.

Keywords

Autonomous research agents, evidence verification, selective prediction, scientific workflows

ACM Reference Format:

Anonymous Author(s). 2027. TrustScientist: Evidence and Branch Revalidation for Autonomous Research Agents. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 9 pages.

1 Introduction

When an autonomous research agent revises its evidence, it must also reconsider the decisions made from that evidence. A certificate may be refreshed while an earlier PROCEED, REFINE, or PIVOT still directs the workflow. *Claim or certificate validity is not decision validity*. The checks supporting a claim and the action selected from those checks have different lifetimes.

False success in agents is already documented [2]. Evidence-based judge evaluation [17] and rubric-guided iterative verification [16] address related failures. For a research agent, verification

also has to govern continuation: when do artifacts justify a reported result, when should the agent acquire more evidence, and which earlier branch should be revisited after a file changes? Even an authentic, correctly quoted output can refer to the wrong metric or task. Updating that output alone leaves the historical action unresolved.

TrustScientist binds represented requirements to admissible artifacts and executable checks. An Evidence-Obligation Graph records their dependencies, and a blocking frontier identifies checks that acquisition can address. Every represented mandatory predicate must pass before the claim closes. Counterfactual Branch Revalidation (CBR) then maintains the decision history. It persists each decision’s inputs and policy context, replays affected branch points, and invalidates the continuation from the earliest changed action. The agent returns to the checkpoint immediately before that decision (Figure 1).

We evaluate two forms of branch history: controlled updates to artifacts from 45 CORE-Bench [14] capsules, and scripted analysis trajectories that fit models, revise evidence, and execute the branch after rollback. The latter connects maintenance to corrected tables, charts and reports while charging receipt verification and continuation work. A generic dependency-replay comparator shares the same tools and matches correctness. Shared-tool acquisition, semantic gates and fixed-retrieval SciFact assess different questions about action selection and answer quality.

The paper makes two contributions:

- (1) **Decision-history revalidation.** CBR records decision inputs, policy context, and checkpoints, then selectively replays affected decisions to recover the earliest changed action and roll back its continuation.
- (2) **A versioned evidence contract and controlled evaluation.** Artifact-bound requirements support certificate and branch revalidation. Separate studies report replay agreement, certificate maintenance, acquisition, and semantic errors, including ties and negative results.

Code, recorded outputs, and offline reproduction scripts are available in the artifact repository.¹

2 Related Work

Incremental build systems track changed inputs and dependent computations [11]; self-adjusting computation studies the consistency of change propagation under mutated inputs [1]. CBR applies these established ideas to persisted research decisions and registered branch outputs. The added interface records the policy context needed to replay an action and the checkpoint needed to discard its continuation. Provenance models record entities, activities, and their relations [18], while program verification connects assertions



This work is licensed under a Creative Commons Attribution International 4.0 License.

¹GitHub: TrustScientist artifact. The Technical Appendix includes an anonymous mirror and reproduction instructions.

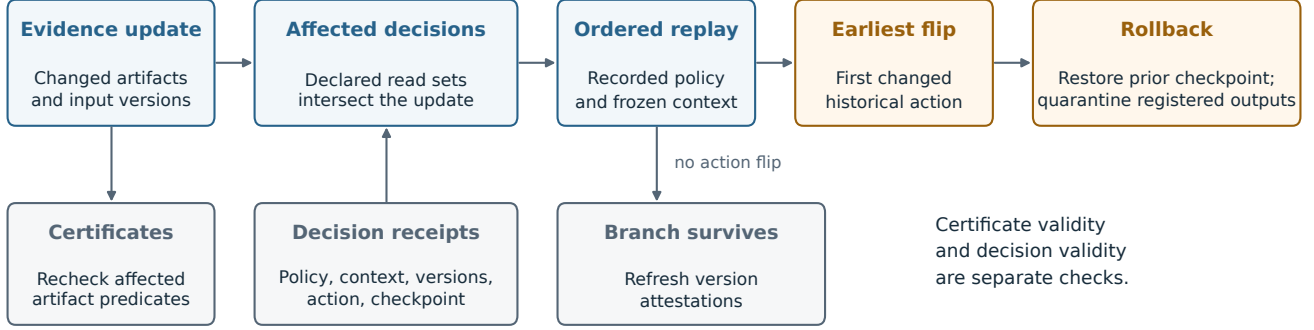


Figure 1: Evidence and decision history require separate revalidation. An evidence update refreshes affected artifact checks and certificates. CBR uses persisted decision receipts to identify affected historical decisions, replays them in order, and rolls back before the earliest action flip, quarantining the registered outputs of the invalid suffix. If all affected actions remain unchanged, the branch survives with refreshed version attestations. Replay uses the recorded policy and context.

to executable conditions [8]. TrustScientist uses these foundations to specify what an evidence certificate covers and when it expires.

The closest research-agent systems also go beyond source tracking. Auditable Autonomous Research (AAR), introduced by Rasheed et al. [13], measures claim-level auditability in deep research outputs. ScientistOne’s Chain-of-Evidence defines claim types and required evidence chains, checking claim–evidence and method–code alignment [10]. ClaimGarden maintains versioned claim states, revalidates them after updates, and controls manuscript and next-action gates [12]. Our focus is the historical action that may survive a refreshed claim state. CBR exposes receipts, ordered replay, and rollback alongside typed artifact predicates and OPEN, CLOSED, and UNVERIFIABLE requirement decisions. The experiments use matched manifests and artifact-acquisition loops; a head-to-head performance comparison with these systems remains outside their scope.

Agent false success [2], REFLECT’s evidence-based judges and controlled trace interventions [17], and DeepVerifier’s rubric-guided verification and iterative feedback [16] motivate checks on both evidence and stopping decisions. Our structured comparator shares the local tool interface and isolates action selection within it; it is not a reproduction of either complete system. Cost-sensitive hypothesis discrimination has a Bayesian active-learning formulation [7]. Our acquisition pilot instead uses model-generated weights and outcome forecasts, without that formulation’s probabilistic assumptions or approximation guarantees.

Scientific fact verification distinguishes support, refutation, and insufficient evidence. SciFact evaluates these relations with scientific abstracts [15]; Atanasova et al. [5] study withholding decisions when crucial evidence is missing. Selective prediction and risk-controlling prediction sets describe the resulting error–coverage tradeoff [3, 6]. TrustScientist uses this perspective for predicates requiring semantic judgment, with empirical thresholds rather than a distribution-free guarantee.

Claim decomposition can ease inspection while discarding context or changing meaning, as Hu et al. [9] document. A finer decomposition can still omit a necessary obligation. We represent

that failure through the distinction between $R(c)$ and $R^*(c)$. The artifact study starts from benchmark task targets and evaluates verification after requirements have been specified.

3 Evidence and Branch Revalidation

TrustScientist maintains two records: a certificate of the current evidence checks and a receipt of each historical decision. The certificate identifies the represented obligations, their artifact bindings, and the checks that passed. The receipt lets CBR reconsider an action when those inputs change. Failed checks remain linked to their originating requirements during acquisition and re-audit.

3.1 Requirements, evidence, and closure

An input consists of a claim c , a represented mandatory requirement set $R(c)$, and an artifact state E . Optional observations are stored outside $R(c)$. Each requirement stores its claim or rubric origin, admissible artifact types, binding rule, and predicate. Artifact records identify content, location, version or digest, and execution metadata. A value match satisfies its numeric predicate; repetitions and baseline comparisons require their own evidence.

The four requirement types organize what must be checked. *Authenticity* (AUTH) checks provenance and agreement between a reported value and its artifact. *Relevance* (REL) checks the claim–artifact relation, including the task, dataset, split, metric, or local question. *Sufficiency* (SUFF) checks specified evidence coverage, such as required replications, uncertainty estimates, or baselines. *Discriminateness* (DISC) checks whether the evidence separates the claim from a named alternative. The latter two require an operational test supplied by the claim or rubric. The type names alone do not define a universal standard of scientific adequacy.

A requirement is CLOSED when admissible evidence is bound and its predicate passes. It is OPEN when evidence is missing or a check fails and an evidence action can address the gap. It is UNVERIFIABLE when the representation or available verifier cannot reliably decide the predicate. A missing expected file can be OPEN; an uninterpretable semantic relation can be UNVERIFIABLE. Both

prevent claim closure. In the CORE protocol, the 17 unbound requirements are recorded as OPEN throughout this paper, matching the experiment’s operational treatment.

Every represented mandatory requirement must be CLOSED before the claim closes. Optional findings remain visible without blocking it. The certificate records the claim and requirement-set version, artifact bindings, predicate outcomes, verifier versions, and unresolved reasons. It attests that $R(c)$ passes those checks on the inspected artifact versions.

3.2 Evidence acquisition and certificate revalidation

An Evidence-Obligation Graph $G = (V, D)$ connects executable checks for the represented requirements. An edge $u \rightarrow v$ means that v depends on the evidence established by u . Each node stores a predicate, artifact bindings, their content digests, a state, and an action specification. A certificate expires when a bound artifact changes. Our reporting implementation uses six nodes: access, integrity, task binding, extraction receipt, literal quotation, and semantic adequacy. Access precedes integrity and binding; both precede extraction, followed by quotation and semantic checks. The six-node graph is an authored, executable reporting schema.

At step t , the blocking frontier is

$$\mathcal{B}_t = \{v : s_t(v) \neq \text{CLOSED}, s_t(u) = \text{CLOSED} \forall u \in \text{pa}(v)\}. \quad (1)$$

Actions address a frontier failure or refresh several affected nodes together. The controller ranks affordable actions by an estimated number of failed checks repaired per cost unit:

$$a_t \in \arg \max_{a: k(a) \leq b_t} \frac{\widehat{g}(a, s_t)}{k(a)}, \quad b_{t+1} = b_t - k(a_t). \quad (2)$$

The gain estimate comes from a fixed tool-effect map; it never reads reference answers. The ranking is a greedy heuristic. After execution, the implementation audits the full small graph again and stores the new evidence digest. An unchanged unsuccessful action is suppressed. Budget exhaustion produces an OPEN result with outstanding checks.

Certificate updates. For directly affected checks I_t , the existing tracker invalidates $U_t = I_t \cup \text{Desc}_G(I_t)$ and rechecks them in topological order. With complete dependencies and deterministic predicates, unaffected checks remain unchanged and induction gives full-revalidation equivalence. The appendix supplies the implementation details. Its outputs are refreshed certificates; historical branch actions are maintained by CBR below.

3.3 Counterfactual branch revalidation

CBR records enough information to reconsider an action after its evidence changes. Each persistent decision receipt contains $d_i = (\pi_i, x_i, W_i, a_i, k_i)$: a policy version, frozen non-evidence context, bound input versions, old action, and checkpoint before the decision. The context includes the retry budget; replay must not consume it again. Receipts also register branch ancestry and local output paths. The implemented history is linear; its suffix is the control-dependent branch.

Algorithm 1 CBR for a registered linear decision history

Require: Receipts $d_{1:n}$, updated evidence E' , replay budget b

- 1: Verify receipts and policy versions; recover any pending rollback
- 2: Hash registered inputs; find affected decisions I
- 3: **for** $i \in I$ in historical order **do**
- 4: **if** replay unavailable or $b = 0$ **then**
- 5: Persist blocked state;
- 6: **return**
- 7: **end if**
- 8: $a'_i \leftarrow \pi_i(E', x_i)$; $b \leftarrow b - 1$
- 9: **if** $a'_i \neq a_i$ **then**
- 10: Persist invalid suffix and rollback intent
- 11: Quarantine registered outputs; restore k_i ;
- 12: **return**
- 13: **end if**
- 14: Refresh witness-version attestation
- 15: **end for**
- 16: Persist surviving history

Let Δ contain changed evidence and W_i include the transitive evidence read set. Counterfactual Branch Revalidation (CBR) computes

$$I = \{i : W_i \cap \Delta \neq \emptyset\}, \quad j = \min\{i \in I : \pi_i(E', x_i) \neq a_i\}. \quad (3)$$

If the minimum is empty, the branch survives. Otherwise CBR invalidates the historical suffix beginning at j , quarantines its registered outputs, and restores k_j . Replay stops at the first flip, before any later decision in the discarded history. Changed witnesses whose actions remain unchanged receive refreshed version attestations. Missing inputs, unsupported policy versions, or exhausted replay budgets block continuation.

With deterministic frozen policies, complete read sets, quiescent inputs, and sufficient budget, unaffected decisions keep their actions. Ordered replay of the affected decisions therefore finds the same first flip as full historical replay. This conditional consistency follows the established change-propagation and trace-consistency perspective [1, 11]. Receipts and checkpoints extend the certificate interface to the registered decision history. Statistical risk, unregistered effects, and the policy’s scientific validity lie outside this consistency argument.

3.4 Scope of closure and verification errors

For a fixed interpretation and scientific scope of c , let $R^*(c)$ denote all obligations needed to justify it. Requirement generation may produce an incomplete set, $R(c) \subsetneq R^*(c)$. Even if the verifier accepts no false represented requirement, a necessary omitted obligation can remain unsatisfied. Excess or malformed requirements create additional utility and specification problems; they are not resolved by the closure rule either.

Let $a_r = 1$ mean that requirement r is CLOSED, $A = 1$ mean that every member of $R(c)$ is CLOSED, and z_r indicate true satisfaction. Define F_{claim} as selecting a claim for which some obligation in $R^*(c)$ is unsatisfied. Let F_{omit} be the part of this event caused by an unsatisfied obligation missing from $R(c)$, and let $F_r = \{a_r = 1, z_r = 0\}$ denote false acceptance of represented requirement r . For a fixed

claim and requirement schema, every full-claim failure involves an omission or an accepted false represented requirement. Hence

$$\Pr(F_{\text{claim}}) \leq \Pr(F_{\text{omit}}) + \sum_{r \in R(c)} \Pr(F_r). \quad (4)$$

The union bound requires no independence assumption. For varying schemas, it applies conditionally and can then be averaged over them. Component error budgets control represented requirements; the omission term requires a separate assessment of completeness. Conditional selective risk divides the unconditional failure probability by nonzero selection coverage. Our experiments leave omission risk and distribution-free component bounds unestimated, so the certificate remains relative to $R(c)$.

Statistical risk qualification. Risk certification requires bounds valid for the evaluated policy. If $q_{r,T}$ bounds false acceptance of requirement r conditional on the adaptive history at stopping time T , the union bound gives represented-set risk at most ϵ when $\sum_r q_{r,T} \leq \epsilon$. Static calibration does not supply such history-conditional bounds. Alternatively, freezing the entire adaptive episode policy allows calibration of its population selective error on independent episodes, with simultaneous bounds for finite-policy selection [4]. The appendix gives a sufficient construction. Both routes require evidence beyond our data and leave the omission term in Equation 4 uncontrolled.

The prototype checks a historical fixed-policy calibration upper bound against $\epsilon = 0.10$. Calibration fails, so it issues no risk-qualified closure. Candidate closure without the guard is reported separately. The executor reads and extracts existing artifacts; experiment and replication actions remain specifications.

Execution scope. Authenticity and relevance use source hashes, task identifiers, extraction receipts, and literal evidence bindings. A semantic rubric checks the requested metric, dataset and method, row or column, aggregation, context, and unresolved conflicts. Sufficiency and discriminativeness need claim-specific tests for experimental adequacy.

The archived CORE extractor scans admissible files in sorted order for a reference value and keeps its first match. Its comparators evaluate supplied manifest predicates. The acquisition controller instead receives a question and public artifacts, reserving reference values for scoring. We report these protocols separately because the former starts with reference-assisted bindings and the latter must acquire its evidence.

4 Experiments

We first evaluate historical branch revalidation, then certificate lifetime. Acquisition, artifact compliance, and semantic verification follow with their own inputs, comparators, and denominators. Table 1 shows saved examples of recovery, stale certificates, a branch flip, and a semantic binding failure. The Technical Appendix supplies the complete receipts and source selectors.

4.1 Historical branch revalidation

Can replay recover a usable continuation after evidence changes? We execute a scripted analysis agent on Iris, Wine, Breast Cancer and Digits. Three split seeds and three search lengths yield 36

histories with 4, 12 or 24 decisions. Each history fits distinct candidate configurations, checks held-out admissibility and baseline comparison, and produces a table, chart and report. Six paired updates per history give 216 episodes: unchanged content, early and late executed configuration revisions, a training-score substitution, candidate deletion, and a revised baseline. Updates are controlled; the trajectories are executed analyses, not natural autonomous AutoResearchClaw histories.

Full replay, CBR and a generic flat dependency-replay comparator share the policy, verified file versions, receipt checks, output quarantine, checkpoint restoration and continuation executor. The generic planner scans each receipt’s read table. Certificate-only refresh updates products without replaying actions. After rollback, the executor refits inadmissible trials, schedules a bounded configuration revision after a failed comparison, and regenerates products. Each replay method executes 135 continuation fits. A separate scorer recomputes macro-F1 from saved predictions, derives the earliest changed action, and inspects checkpoints, quarantined files and final table/chart/report consistency.

CBR matches full replay at every rollback point, retains all 114 non-flip branches, and leaves no stale final action. Total post-update policy entries fall by 35.3% (95% paired cluster-bootstrap interval [32.2, 38.6]); the 12- and 24-decision histories save 36.2% and 37.4%. Two order-balanced timing repetitions show a smaller 10.3% reduction [9.3, 11.2] in post-update wall time, including refitting and file operations. Intervals resample the 12 dataset-split clusters, not individual mutations. The generic comparator matches correctness and uses fewer calls: its per-receipt check skips some current attestations revisited by CBR’s union of changed files. These results support selective replay relative to full replay, not superiority over generic dependency systems or ClaimGarden.

The original CORE study remains separate: 405 controlled updates to 45 capsules produce 254 branch flips across 711 historical decisions. CBR matches all full-replay checkpoints with 382 versus 416 replay evaluations; the certificate-only tracker retains 326 changed historical actions. The appendix preserves that protocol, the new executed trace and cost details.

4.2 Certificate revalidation

We compare executable dependency tracking with a stateful, carefully prompted validator on copies of 124 source files underlying 79 archived reporting states from 45 CORE capsules. Of these files, 39 are shared by questions within 21 capsules. Each capsule has a declared dependency schema linking source versions, extraction receipts, bindings, and closure; 55 initial literal contracts pass. Nine controlled events per capsule change or restore a source, receipt, report, or source–receipt pair, or change an unrelated file. The original artifacts remain untouched.

All policies receive the same dependency metadata, changed-file identifiers, checker, and per-event budget, sufficient for full revalidation. Full re-audit skips unregistered-file changes and otherwise checks every node. The stateful Qwen3-8B validator receives its persistent cache, complete event history, and an explicit transitive-invalidation algorithm. The graph policy traverses an index; the flat-table comparator obtains the same affected set by repeated scans. A malformed LLM plan falls back to full re-audit. All 405

Table 1: Recorded examples from the evaluated studies, selected after inspecting saved outputs. Version and branch changes are controlled interventions; acquisition reads published artifacts. The cases illustrate recovery, maintenance, and a binding failure; they provide no frequency estimate. Source IDs and receipts are in the Technical Appendix’s Recorded Case Studies.

Case	Evidence change and TrustScientist response	Paired comparator and boundary
Two-step recovery SVM F1 on Musk1+	Wrong capsule binding and interrupted extraction keep the report OPEN. Rebind, then resume extraction; re-audit closes the unguarded report with the quoted answer 87.5155.	The structured selector also closes correctly after two actions. Risk-guarded ECC remains OPEN because calibration fails.
Shared-source update Three reporting chains	A changed source invalidates all three downstream certificates. Dependency re-audit checks ten affected nodes and removes all stale closures.	The stateful Qwen validator updates one chain, retaining two stale closures. Flat tracking matches the graph; full re-audit checks eleven nodes.
Historical branch flip Two recorded decisions	Replay leaves the earlier REFINE unchanged, but changes the later PROCEED to REFINE. CBR invalidates the later continuation and rolls back before it.	Full replay chooses the same checkpoint. The certificate-only tracker updates predicates but retains the old PROCEED. No replacement experiment is executed.
Wrong-metric binding Random-classifier macro-F1	The file, digest and supplied relation flag pass, although the quoted value belongs to neural-network macro recall. TrustScientist incorrectly closes.	GPT-5.6-Sol keeps the manifest OPEN and identifies the metric and classifier mismatch.

Table 2: Executed controlled histories. Exact counts earliest rollback matches on 102 flip episodes; Final counts consistent final actions and products on all 216 episodes. Calls includes replay, receipt verification and continuation policy entries, excluding common initialization.

Policy	Exact /102	Final /216	Calls
No re-audit	0/102	59/216	0
Certificate-only	0/102	114/216	0
Full replay	102/102	216/216	4,334
Generic replay	102/102	216/216	2,709
TrustScientist-CBR	102/102	216/216	2,805

formal model requests complete without transport failures or fallbacks.

An independent evaluator reconstructs file bytes after each event and checks the certificates’ declared input versions. Each method makes 711 certificate decisions: 332 contracts are valid, 163 initially passing contracts have become stale, and 216 were initially non-passing. Stale acceptance uses 163 as its denominator; valid retention uses 332. Both metrics refer to the version contract.

Dependency tracking matches full revalidation with 47.0% fewer check evaluations (Figure 2). The 95% paired capsule-bootstrap interval is [42.2, 51.8]. The LLM retains 38/163 stale certificates (23.3%) and loses five valid closures. Its invalidation set is exact on 227/405 events; 93 events omit required checks. The shared-source example in Table 1 leaves two LLM closures stale.

The flat dependency table matches the graph’s decisions and check count exactly. Selective re-audit gains from executing the dependency contract; either data structure can supply the affected set.

4.3 Evidence acquisition

Acquisition can recover unresolved reporting states, but its usefulness depends on both the available actions and the stopping gate. We compare the action selector with a structured LLM using the

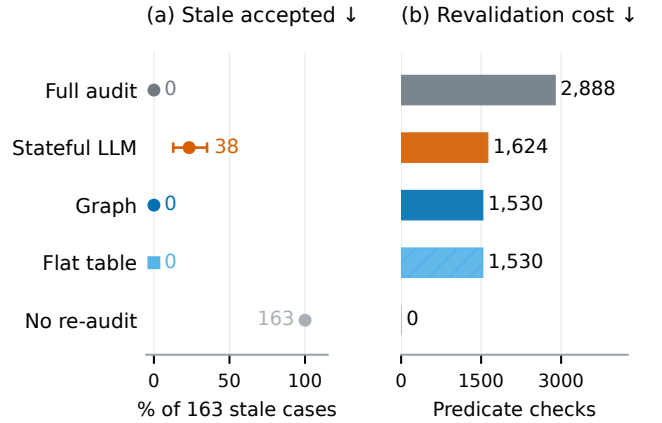


Figure 2: Controlled version updates. Counts beside points are stale certificates accepted out of 163; whiskers show capsule-bootstrap 95% intervals. Graph and flat tracking, like full re-audit, retain all 332 valid certificates; the LLM retains 327. No re-audit retains all 332 valid and all 163 stale certificates. Check counts exclude initialization.

same tools. The frozen protocol separates natural retrieval difficulties from controlled interacting faults, with reference answers reserved for final evaluation.

Tasks, policies, and measurement. Fifteen capsules, selected by a fixed hash order, are reserved for historical calibration. Evaluation comprises 46 natural questions from 30 capsules and 36 controlled states drawn from 12 nominal-clean requirement bundles in seven capsules. Controlled scenarios pair wrong task binding with an interrupted extraction receipt, pair stale integrity metadata with an interrupted receipt, or change a digest after an initially passing check. The last scenario creates a stale decision deliberately.

All policies share Qwen3-8B, source snapshots, tools, structural checks, and a six-condition semantic rubric. Actions read or search

Table 3: Candidate outcomes before the risk guard, budget 4. Columns show correct and incorrect candidate closures; denominators are all 46 natural or 36 controlled attempts. ECC’s final risk-qualified closures are zero. Only w/o risk guard and Structured LLM turn these candidates into final closures. The postcheck-change scenario is included in controlled states.

Policy	Natural ($n = 46$)		Controlled ($n = 36$)	
	Correct	Wrong	Correct	Wrong
ECC	21	9	22	2
w/o graph	21	9	11	1
w/o risk guard	21	9	22	2
w/o budget selection	21	9	22	2
w/o re-audit	18	4	0	12
Structured LLM	21	9	22	2

published outputs, refresh a source, repair task binding, or resume extraction. Budgets are 1, 2 and 4 declared units, with at most four actions. Rebinding and resumed extraction cost one unit, search/fetch two, and broad refresh three. Tokens are logged separately from these policy costs. The executor acquires existing artifacts rather than running new science.

The six policies comprise the full evidence-closure controller (ECC), four ablations, and a structured LLM selector. The selector receives the same graph-filtered candidate actions, checks, and prior receipts, and shares the final predicate aggregator without the calibration guard. The graph and obligations are thus supplied to both selectors. Removing the graph replaces it with a flat failure list and fixed scheduling priority. Removing budget-aware selection prioritizes broad refresh and stops when it is unaffordable. Removing re-audit retains cached predicates after evidence changes. The graph ablation also changes ordering.

False termination and successful closure divide invalid and valid final closures, respectively, by all attempts. Selective risk divides errors by closed attempts and is undefined when none close. Recovery success uses initially unresolved predicate states as its denominator. Actions-to-closure and cost-to-closure are conditional means over successful closures; unconditional acquisition cost is reported alongside them. The operational evaluator combines artifact checks, literal binding, and benchmark answer matching. It shares the controller’s structural and literal-binding functions. We retain its frozen labels while distinguishing reference agreement from source fidelity: a faithfully quoted output can differ from the reference target.

Calibration and control outcomes. Historical calibration accepts seven reserved capsules, including two automatically scored errors. The one-sided 95% Clopper–Pearson upper bound is 0.659, exceeding the preset 0.10 threshold. The risk guard rejects every candidate. ECC and its guarded ablations therefore produce zero false terminations and zero successful risk-qualified closures, with undefined conditional selective risk. Calibration has failed.

The full factorial contains 1,476 completed policy rollouts, 435 distinct model requests shared through caching, and 807 executed

tool actions. Thirteen unusable replies, including three request failures, remain unresolved.

Without the risk guard, budget-4 control produces 21/46 successful natural closures (capsule-bootstrap 95% interval $[30.9, 59.6]\%$) and 9/46 false terminations. Controlled outcomes are 22/36 successful closures (interval $[52.4, 66.7]\%$) and 2/36 false terminations. The structured selector matches both counts and every paired closure/correctness outcome: 82/82 agree. Natural cases lack source snapshots for restoration, leaving acquisition often limited to repeating a search with the same question at a later offset.

At budget 4, removing the graph reduces correct controlled candidates from 22 to 11. Omitting re-audit leaves 12 stale, invalid candidates and no correct controlled candidates. At budget 2, cost-aware selection produces 11 correct controlled candidates, while broad-refresh priority produces none. The graph contrast includes the changed ordering; the budget contrast includes stopping when the preferred refresh is unaffordable.

Natural recovery reaches 3/24 initially unresolved tasks without the guard. Raising the budget from 2 to 4 adds two false terminations and no correct closures. Saved failures include a processed-data row cited for an analysis-script question and a regression estimate cited as a sample count. Literal quotation misses these context and value-type errors. The appendix distinguishes them from reference-output disagreements and records all budget and cost denominators. These utility outcomes precede the failed risk guard.

Executed trace. The two-step recovery in Table 1 links each executed action to a new evidence digest, receipt, and re-audit. The benchmark-matching candidate closes only without the failed calibration guard; the appendix preserves the complete trace.

Disjoint acquisition pilot. The frozen pilot uses 41 questions from 24 newly acquired public-train capsules, disjoint from earlier cohorts. Both selectors share model-generated hypotheses and probes, accumulated evidence, artifact tools, a four-unit budget, and the final gate. Our heuristic maximizes predicted pairwise disagreement per cost; the validator follows the model’s choice. Across 623 requests, each closes 14 correct and 11 incorrect answers. Every outcome matches, and neither repairs any of the 11 initially wrong answers. The heuristic uses 4.4% more tokens.

The validator recommends 59 already executed queries, triggering fallbacks. A post hoc interface repair restricts choices to valid actions. Both policies then close 13 correct answers; the heuristic abstains on one additional wrong answer. The appendix gives the full comparators, costs, and diagnostics.

4.4 Artifact contract enforcement

The archived artifact protocol starts with candidate bindings and applies controlled manifest violations. A restoration test returns each record to its known clean state. A separate fresh-workspace diagnostic reruns public capsules; its outcomes remain separate from the acquisition study.

Requirements and candidate retrieval. We use the 45 capsules in the CORE-Bench test manifest: 18 computer science, 14 social science, and 13 medical science tasks; 22 use Python and 23 use R. Their shipped results/ directories provide the nominal clean artifacts. Manually authored benchmark questions and reference

Table 4: Reference-assisted candidate retrieval. Field and language panels describe the same 79 requirements; they are not additive.

Subset	Candidates/total	Coverage (%)
Computer science	29/36	80.6
Social science	21/24	87.5
Medical science	12/19	63.2
Python	34/42	81.0
R	28/37	75.7
Overall	62/79	78.5

outputs define the targets. Merging repeated reference executions leaves 79 unique capsule–question requirements.

The evaluator locates reference values in extracted artifacts and marks the clean records’ claim–artifact relations as valid before constructing corrupt controls. Candidate retrieval is therefore reference-assisted. A matching value locates a possible binding, whose semantic accuracy still requires checking.

Candidates are retrieved for 62/79 requirements (78.5%; capsule-bootstrap 95% interval [66.7, 88.1]). Table 4 includes all requirements in the denominator, with the other 17 remaining OPEN. Coverage ranges from 12/19 in medicine to 21/24 in social science, reflecting the extractor’s coverage of the shipped artifacts.

Matched-manifest decision policies. For each retrieved candidate, five controls change one condition: a missing artifact, an inadmissible type, a path outside the permitted results namespace, a stale digest, or binding to the wrong local question. This yields 62 clean and 310 corrupted states. Only the decision records are mutated; the original bundles are retained.

Both LLM baselines receive the claim, requirement, extracted value, manifest fields, execution status, and semantic evidence supplied to the closure policy. Their inputs include precomputed type, path, digest, and relation checks, but exclude raw files and images, TrustScientist’s verdict, and corruption labels. The saved prompt requests a closure judgment and reason. Evidence access is matched; the LLM procedure makes a joint judgment where TrustScientist enforces a predicate conjunction.

DeepSeek-V4-Flash closes 55/62 clean and 149/310 corrupted states (Table 5). GPT-5.6-Sol closes 59/62 clean states and rejects all 310 corruptions. TrustScientist closes all 62 nominal clean states and rejects all corruptions. *Residual corrupted closure* uses every corrupted state in its denominator, independently of whether the policy closed its clean counterpart.

The three GPT nonclosures identify metric, task, or context mismatches. Table 1 shows one: a value passes the supplied relation flag but belongs to another metric and classifier. TrustScientist accepts that flag and closes incorrectly. Nominal clean closure and correct semantic binding are therefore different outcomes; these GPT abstentions have no established error label.

GPT enforces every tested corrupted-state constraint. A generic validator could also implement the same conjunction. Because the control construction supplies the type labels, their independent inference is outside this protocol.

Table 5: Matched-manifest compliance on retrieved candidates. Lower corrupted closure is better. Each policy receives the same evidence fields; the LLMs use their saved fixed-prompt configurations.

Policy	Nominal clean closed	Corrupted states closed
DeepSeek-V4-Flash	55/62	149/310
GPT-5.6-Sol	59/62	0/310
TrustScientist	62/62	0/310

Restoration and fresh execution. All 310 controlled violations remain unresolved. Restoring each known clean record after naming its first failed gate makes all 310 close on re-audit. This transition exercises restoration under a known intervention.

The fresh-workspace diagnostic covers 20 public-train capsules and 59 task questions. In each of five runs, 19/20 tasks meet their reference targets and 58/59 questions match. Mean summed task wall time per run is 765.9 seconds. Each run starts with a fresh workspace and the same prepared host-specific compatibility repairs. The archived CUDA 10.1 program fails on the Ampere GPU in every run; its CPU fallback also misses the reference target. That task stays in every denominator. The appendix records the per-run outcomes.

4.5 Semantic verification

Paired semantic gates. We hold retrieved context and reports fixed on 79 previously inspected questions while varying the semantic gate, using 237 Qwen3-8B requests. An independent scoring implementation measures benchmark-answer agreement. The literal contract closes 44 correct and 11 wrong answers. A structured rubric closes 35 and four; blind answer agreement closes 34 and six. Adding scope witnesses closes 25 and five. It rejects one error but loses nine correct closures, a success difference of −11.4 percentage points (95% capsule-bootstrap interval [−19.8, −2.9]). Additional witnesses reduce useful closure in this paired study. Complete gate definitions and scores are in the appendix.

Fixed-retrieval SciFact. SciFact evaluates relation resolution under fixed retrieval. A pair is CLOSED when its SUPPORTS or REFUTES relation is resolved. The corpus contains 209 evidence-bearing pairs and 130 NEI controls. TrustScientist correctly closes 98/209 evidence-bearing pairs and falsely closes 11/130 NEI pairs. Its 118 closures also include nine SUPPORTS/REFUTES reversals.

NEI-FCR divides false closures by NEI cases. Coverage divides all closed decisions by all cases, and valid-claim recall divides correctly resolved evidence-bearing pairs by 209. Macro-F1 treats abstention as NEI; selective risk divides all wrong relations by closures. The appendix retains the complete operating-point table and repeated-run comparisons. The matched-coverage curves show the cost in useful relation resolutions that can accompany lower NEI-FCR.

5 Discussion and Limitations

The branch experiment exposes the distinction between a current certificate and a current decision history. Certificate revalidation

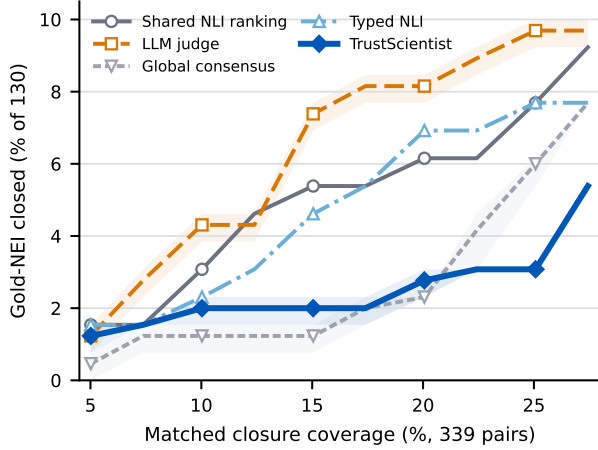


Figure 3: Original fixed-retrieval SciFact matched-coverage curves: five-run means, with shading spanning runs for LLM rankings. The ordinate is closed gold-NEI pairs divided by 130; relation reversals are excluded. Shared NLI denotes direct NLI, rule, and risk-controlled policies. Clipped-score ties retain dataset order, limiting ranking comparisons.

refreshes evidence checks; CBR revisits the action selected from them. Its conditional consistency argument relies on frozen deterministic policies, complete declared read sets, quiescent inputs, and sufficient replay budget. An ordinary dependency tracker with the same receipt, replay, and rollback operations can implement CBR. Change propagation and control-dependence tracking remain its established foundations. The flat-table tie likewise places the certificate benefit in dependency enforcement rather than in the graph data structure.

The histories and update events are constructed. They measure maintenance under the registered contract, leaving natural branch-error prevalence, arbitrary LLM redecision, and unrestricted autonomous replacement-branch execution open. The longer-history executor performs bounded model refits and publication updates under a scripted policy. PIVOT parsing and rollback are separate functional checks, rather than observed scientific pivots. The Stage-15 adapter binds the judge’s refreshed verdict; correctness of that verdict and undeclared upstream scientific dependencies require separate validation. Rollback can quarantine registered local outputs, while external jobs and unregistered effects remain beyond its reach. The new study charges policy evaluations during replay, receipt verification and continuation. Balanced wall-time measurement includes file operations and refitting, but excludes common history initialization. This local CPU result is not a general end-to-end cost guarantee.

Requirements come from benchmark tasks and an authored graph, with no new human annotations. Reference answers support benchmark scoring but leave the completeness of $R(c)$ and the accuracy of semantic bindings unresolved. The 62 reference-assisted candidates have no independent expert binding audit. The manifest experiment supplies validity flags and constructed type labels; it evaluates enforcement under those inputs. Its controlled

faults exclude natural error prevalence, partially sufficient experiments, and interacting failures. The restoration test uses known clean records, leaving autonomous diagnosis and artifact repair unmeasured. Fresh-workspace runs reuse a task subset and prepared repairs: they show execution repeatability, not independent populations or autonomous repair. The shipped-bundle audit also leaves full rerunning of all 45 capsules outside its scope.

Semantic errors remain after literal quotation, answer agreement, scope witnesses, and the tested acquisition heuristic. The shared-tool selectors tie; neither acquisition experiment demonstrates a correction advantage, and the tested extensions show no advantage over the structured comparators. The original acquisition evaluator shares structural and literal-binding functions with the controller. Separate code in the paired gate study improves implementation independence while using the same benchmark semantic gold. Restricted actions, one model, small cohorts, proxy costs, and coupled graph/budget ablations limit generalization and attribution. The disjoint public-train pilot broadens task coverage, though its interface repair is post hoc. The new generic dependency-replay comparator matches branch correctness and uses fewer evaluations; frontier-model redecision remains untested. SciFact illustrates the tradeoff between lower NEI false closure and retaining correct relations, with no uniform dominance across the matched-coverage curves.

Calibration fails the historical risk threshold and rejects every candidate. Zero errors with zero closures provide no useful risk-control result. History-conditional bounds or independent calibration of a frozen adaptive episode policy would be needed for adaptive risk certification, and either would still leave omitted obligations uncontrolled. Repeated mutations and seeds cannot replace unused tasks. The experiments establish consistency of recorded checks and decisions within their contracts; scientific or publication acceptance requires broader evidence.

6 Conclusion

TrustScientist maintains versioned evidence certificates and the historical decisions made from them. CBR identifies affected decisions, replays them in order, and restores the checkpoint before the earliest changed action. On the controlled histories, it recovers every full-replay rollback point with fewer post-update policy evaluations and supports executed model refits after rollback. The matched generic replay result shows that this benefit is available to ordinary dependency systems with decision replay. Complete natural trajectories and semantic policy validation remain the next tests of practical value.

References

- [1] Umut A. Acar, Matthias Blume, and Jacob Donham. 2011. A Consistent Semantics of Self-Adjusting Computation. arXiv:1106.0478. doi:10.48550/arXiv.1106.0478
- [2] Laksh Advani. 2026. From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents. FAGEN workshop at ICML 2026. arXiv:2606.09863 doi:10.48550/arXiv.2606.09863
- [3] Anastasios N. Angelopoulos and Stephen Bates. 2023. Conformal Prediction: A Gentle Introduction. *Foundations and Trends in Machine Learning* 16, 4 (2023), 494–591. doi:10.1561/22000000101
- [4] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. 2021. Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control. Version 5, revised 29 September 2022. arXiv:2110.01052 [cs.LG] doi:10.48550/arXiv.2110.01052
- [5] Pepa Atanasova, Jakob Grue Simonsen, Christina Lioma, and Isabelle Augenstein. 2022. Fact Checking with Insufficient Evidence. *Transactions of the Association for Computational Linguistics* 10 (2022), 746–763. doi:10.1162/tacl_a_00486
- [6] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael Jordan. 2021. Distribution-free, Risk-controlling Prediction Sets. *J. ACM* 68, 6 (2021), 1–34. doi:10.1145/3478535
- [7] Daniel Golovin, Andreas Krause, and Debajyoti Ray. 2010. Near-Optimal Bayesian Active Learning with Noisy Observations. In *Advances in Neural Information Processing Systems*, J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta (Eds.), Vol. 23. Curran Associates, Inc., 766–774. <https://arxiv.org/abs/1010.3091>
- [8] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580. doi:10.1145/363235.363259
- [9] Qisheng Hu, Quanyu Long, and Wenya Wang. 2025. Decomposition Dilemmas: Does Claim Decomposition Boost or Burden Fact-Checking Performance?. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Albuquerque, New Mexico, 6313–6336. doi:10.18653/v1/2025.naacl-long.320
- [10] Rui Meng, Bhavana Dalvi Mishra, Jiefeng Chen, Chun-Liang Li, Palash Goyal, Mihir Parmar, Yiwen Song, Yale Song, Rajarishi Sinha, Parthasarathy Ranganathan, Burak Gokturk, Jinsung Yoon, and Tomas Pfister. 2026. ScientistOne: Towards Human-Level Autonomous Research via Chain-of-Evidence. arXiv preprint arXiv:2605.26340. doi:10.48550/arXiv.2605.26340
- [11] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build systems à la carte. *Proceedings of the ACM on Programming Languages* 2, ICFP (2018), 1–29. doi:10.1145/3236774
- [12] Hafumi Nishi. 2026. ClaimGarden: Update-Aware Claim-State Control for AI Scientist Workflows. ICML 2026 AI for Science Workshop. AI Scientist Proposal Competition track. <https://openreview.net/forum?id=Y8INhipPQe>
- [13] Razeen A Rasheed, Somnath Banerjee, Animesh Mukherjee, and Rima Hazra. 2026. From Fluent to Verifiable: Claim-Level Auditability for Deep Research Agents. arXiv preprint arXiv:2602.13855. doi:10.48550/arXiv.2602.13855
- [14] Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. 2024. CORE-Bench: Fostering the Credibility of Published Research Through a Computational Reproducibility Agent Benchmark. arXiv preprint arXiv:2409.11363. doi:10.48550/arXiv.2409.11363
- [15] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. 2020. Fact or Fiction: Verifying Scientific Claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 7534–7550. doi:10.18653/v1/2020.emnlp-main.609
- [16] Yuxuan Wan, Tianqing Fang, Zaitang Li, Yintong Huo, Wenxuan Wang, Haitao Mi, Dong Yu, and Michael R. Lyu. 2026. Inference-Time Scaling of Verification: Self-Evolving Deep Research Agents via Test-Time Rubric-Guided Verification. arXiv:2601.15808 doi:10.48550/arXiv.2601.15808
- [17] Leyao Wang, Yanan He, Peng Chen, Asaf Yehudai, Yixin Liu, Rex Ying, Michal Shmueli-Scheuer, and Arman Cohan. 2026. Time to REFLECT: Can We Trust LLM Judges for Evidence-based Research Agents? arXiv:2605.19196 doi:10.48550/arXiv.2605.19196
- [18] World Wide Web Consortium. 2013. *PROV-DM: The PROV Data Model*. W3C Recommendation. World Wide Web Consortium. Edited by Luc Moreau and Paolo Missier; 30 April 2013. <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>